Technology Stack Report

Prepared per Lab 2 Manual, Section 3.4 (Technology Stack Discussion)

---

(1) Uploaded Material

We uploaded two files to a fresh AI chat session (no earlier conversation history, so the recommendation would be independent): our Lab 1 system description — Doomscroll auto-scrolls on-screen content by tracking the user's head pose and gaze through the webcam — and our Lab 2 class diagram (`class_diagram.jpg`), showing the entity, boundary, and control classes we've identified so far. Both are submitted as their own Lab 2 deliverables, so we haven't reproduced them in full here.

(2) Prompt and AI's Response

We used the prompt suggested in the Lab 2 manual, adding a short note on our team's existing experience as the manual invites:

> *“Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team. Team context: the gaze/head-tracking core is already implemented in Python (OpenCV, MediaPipe, scikit-learn, PyObjC/pywin32); the team has prior experience with Python but not with building a polished desktop application UI.”*

The AI's response, reproduced in full below:

> Based on your system description and class diagram, here's how I'd think about the stack.
>
> ## The core framework decision: Electron, Tauri, or native Python GUI?
>
> **I'd recommend Tauri, with your Python gaze-tracking core running as a sidecar process.** Here's my reasoning:
>
> **Why not a native Python GUI toolkit (PyQt/PySide, Tkinter, wxPython)?** This is tempting because it avoids IPC entirely — your Python code just calls straight into the UI. But your mockups (setup screen, calibration sequence with animated points, always-on-top HUD overlay, settings panel, modal dialogs) describe a genuinely UI-heavy app, and Qt is really the only Python option with the polish to pull that off. Even then, PyQt/PySide has licensing complexity (GPL vs. commercial LGPL considerations depending on how you distribute), and "make it look and feel like a real modern app" is harder in Qt than in a web-based UI unless someone on the team already knows QML/Qt Widgets well. Given you say you don't have desktop UI experience yet, you'd be learning an unfamiliar, fairly idiosyncratic UI paradigm *and* building a polished always-on-top HUD in it — that HUD in particular (a translucent, click-through-ish overlay that sits above other apps) is fiddly in Qt and very well-trodden in

Electron/Tauri.

Why not Electron? Electron is the safe, extremely well-documented choice, and I wouldn't talk you out of it if the team wants maximum tutorial/Stack Overflow coverage. But for this specific app I think it's a worse fit than Tauri:

- You explicitly care about low CPU/memory footprint "to run alongside other apps" — Electron ships a full Chromium + Node runtime per app (100s of MB idle RAM) which directly cuts against that requirement, especially since this app is meant to run continuously in the background while the user does something else.
- You care about a real-time always-on-top overlay running concurrently with a webcam capture loop and screen-content classification — you want your Python process to have CPU/memory headroom, not to be fighting a heavy Electron shell for resources.
- Security posture: you're processing biometric data (face/eye video) and explicitly promising it stays on-device. Tauri's smaller attack surface (Rust backend, no bundled Node in the untrusted renderer, stricter default IPC permissions) is a better story to tell users and reviewers than Electron's historically loose defaults.

Why Tauri specifically works for your architecture: Tauri's whole model is "web frontend (HTML/CSS/JS, e.g. React or Svelte) + a small native Rust core," and it explicitly supports sidecar binaries — external processes the Rust core manages (spawns, monitors, restarts, kills on quit) and talks to over stdin/stdout or local sockets. You'd package your existing Python gaze-tracking core (OpenCV + MediaPipe + scikit-learn + PyObjC/pywin32) as a PyInstaller-frozen executable and run it as exactly this kind of sidecar. That means:

- You do not rewrite your gaze-tracking logic. It keeps running as Python, doing the same OpenCV capture loop, MediaPipe landmark detection, and Ridge regression calibration it does today. You're not porting numerical code to another language.
- The Rust/JS side owns windowing, the always-on-top overlay, dialogs, and settings UI — the stuff you don't have experience with and where a web UI genuinely helps.

IPC between the Python core and the UI

Don't reinvent this — pick one boring, well-understood mechanism:

- WebSocket server in Python (e.g. via `websockets` or a minimal `FastAPI`/`uvicorn` app), local-only (127.0.0.1 or a Unix domain socket/Named Pipe), with the Tauri/JS frontend as the client. This maps naturally onto your class diagram: `GazeEstimationController` pushes `GazePoint`/`GazeVector` events as they're produced (your 15+ fps loop), `AutoscrollController` emits `ScrollCommand` events, `CalibrationController` drives the calibration point sequence and reports progress, and `SettingsController` handles reads/writes of `AutoscrollSettings`. A simple JSON message protocol keeps this decoupled and testable independent of the UI.
- Alternative: raw stdin/stdout with newline-delimited JSON is even simpler and is exactly what Tauri's sidecar API is built around — no port management, no localhost security surface. I'd actually lean

toward this over a WebSocket for a single-consumer local app; use a socket/WebSocket only if you anticipate multiple UI windows/processes needing the stream simultaneously, in which case a lightweight local pub/sub (or just having Python push to Rust, which fans out to Tauri windows via Tauri's own event system) is cleaner.

- Important non-functional constraint: since you're promising no raw video/biometric data leaves the machine without consent, keep this IPC unambiguously local-only (no exposed network interface) and don't let it become a channel that accidentally makes telemetry/crash reporting easy to wire up carelessly later — audit whatever you send in gaze/frame payloads.

Note OS-level scroll injection (PyObjC/pywin32) stays in Python — no reason to move that to Rust; Python already has this solved for both platforms.

Mapping the UI screens onto Tauri

Your boundary classes map cleanly onto Tauri windows/views:

- `SetupScreen`, `CalibrationScreen`, `SettingsScreen` → regular Tauri windows, built as a normal SPA (React or Svelte — either fine, Svelte is lighter and arguably easier to pick up from scratch than React if nobody knows either yet).
- `ReaderOverlay` (the always-on-top HUD) → a separate Tauri window with alwaysOnTop, no decorations, transparent background, and click-through configured on the transparent regions (Tauri supports this natively per-window, which is exactly the "several distinct window types with very different chrome" pattern Tauri is designed for).
- `RecalibrationPromptDialog` → a Tauri dialog/modal window, triggered by an event from `GazeEstimationController` (matches the "triggers" dependency in your diagram) when confidence drops.

State management

- Frontend UI state (current calibration point index, which screen is showing, HUD status indicator): plain React state / Svelte stores is enough — this app's UI state graph is small. Don't reach for Redux/heavy state libraries; it's not warranted here.
- Domain state that's shared across the Python core and multiple windows (current `AutoscrollSettings`, active `CalibrationProfile`, session status) should be owned by Python (source of truth) and treated as read-only/mirrored on the frontend, pushed via IPC events — avoids the two sides disagreeing about state.
- Persisted settings: store `AutoscrollSettings` and `CalibrationProfile` as JSON/SQLite on disk. Simplest: Python already owns this logic (it's already doing the ML), so have Python read/write a JSON file (or SQLite via sqlite3/SQLAlchemy if you want structured querying of `Session` history) in the OS-appropriate app-data directory (the `platformdirs` package handles cross-platform path resolution cleanly). Alternatively Tauri has a tauri-plugin-store if you'd rather the frontend own settings — but given your `SettingsController` is a control-layer class in your own diagram sitting on the Python side conceptually, I'd keep persistence in Python for a single source of truth.

Packaging & distribution (Windows + macOS)

This is the part I'd budget real time for, since it's the area most likely to bite an inexperienced team:

- Freeze the Python sidecar with PyInstaller (--onefile per platform) — but note OpenCV + MediaPipe + scikit-learn pull in a lot of native/compiled dependencies, so test the frozen binary early and often, not just at the end. Build the Python sidecar separately per-OS (PyInstaller doesn't cross-compile) — you'll need a macOS machine/CI runner and a Windows one.
- Tauri bundler produces .dmg/.app for macOS and .msi/.exe (NSIS) for Windows, embedding your sidecar binary per Tauri's sidecar packaging conventions (it wants platform-target-triple-suffixed binaries, e.g. gaze-core-x86_64-apple-darwin).
- Code signing is non-optional in practice here: macOS Gatekeeper will block an unsigned app outright for most users, and an app requesting camera and screen-recording permissions (which yours must, for webcam + screen content capture) is exactly the kind of app macOS scrutinizes hardest — you'll need an Apple Developer account, notarization, and a proper Info.plist/entitlements declaring camera/screen-capture usage with clear purpose strings. On Windows, unsigned executables trigger SmartScreen warnings; a code-signing certificate avoids that friction. Budget for this — it's often underestimated by student teams and can block a release near the deadline.
- CI: GitHub Actions with matrix builds (windows-latest, macos-latest) is the standard way to produce both installers without every team member needing both machines locally.
- Auto-updates: Tauri has a built-in updater plugin (signed update bundles, checks a JSON manifest you host — even a static file on GitHub Releases works) — much less to build yourself than Electron's electron-updater/Squirrel setup. Worth turning on once you're past MVP, given you'll likely be iterating on calibration accuracy post-release.

Other libraries/services worth mentioning

- Crash reporting: Sentry has both a Rust/Tauri SDK and a Python SDK, so you can capture crashes/exceptions from both halves of the app into one dashboard — useful since a bug could originate in either process and you'll want to tell which.
- Testing:
- Python core: pytest for the gaze estimation/regression/content-classification logic — these are the most "correctness-critical" and most testable parts (feed known landmark data in, assert gaze point out), so prioritize coverage here.
- Frontend: Vitest + Testing Library (if React/Svelte) for component logic; Playwright can drive the actual Tauri window for a handful of end-to-end smoke tests (app launches, calibration screen advances through points, settings persist across restart).
- IPC contract: since Python and JS are decoupled processes, write a couple of integration tests that spin up the real Python sidecar and assert on the JSON messages it emits — this is the seam most likely to silently break as either side evolves independently.

- Logging: structured JSON logging from Python (structlog or stdlib logging with a JSON formatter) makes it much easier to correlate a support report against both the Rust/JS side's logs and the Python core's, since you'll otherwise have two separate log streams to reconcile when debugging a user's issue.
- Permissions handling: both macOS (camera + screen recording, the latter requiring System Settings interaction, not just a runtime prompt) and Windows (camera privacy settings) have OS-specific permission flows your SetupScreen needs to detect and guide the user through — this is genuinely one of the fiddlier parts of the whole app and worth prototyping early rather than leaving until the end, since screen-recording permission on macOS in particular requires an app restart after granting, which affects your setup-flow UX design.

Bottom line: Tauri (Rust core + React or Svelte frontend) driving the windows/HUD/dialogs, your existing Python/OpenCV/MediaPipe/scikit-learn core unchanged and running as a PyInstaller-frozen sidecar, talking over stdio or a local-only WebSocket with a small JSON event protocol, Python as source of truth for settings/calibration persistence, and real budget set aside early for code signing and OS permission flows on both platforms — that's the area most likely to surprise a team doing this for the first time.

(3) Our Opinion (Section 3.4.4)

*We're commenting on **Tauri**, the AI's top pick for the application shell. We don't think it's the right call for us this semester: none of us know Rust, and since our gaze-tracking core is already working in Python, something like PySide would let us build the UI with the language and libraries we already know instead of learning a whole new toolchain under a course deadline. Tauri's smaller footprint and stricter security model are real advantages, but they matter much more for a product we'd actually ship to users than for a project we mainly need to demo and get marked on.*